PYTHON

Complete Student Guide

From Basics to GUI — With Real-World Code

Beginner-Friendly | Runnable Code | Real Analogies | Cheatsheet

Topics Covered: Variables • Operators • Control Flow • Functions • OOP • File Handling • Regex • Threads • MySQL • Sockets • GUI

Chapter 1

First Python Program

Hello World, Calculator & Running Code

1. First Python Program

What is Python?
Python is a high-level, interpreted, general-purpose programming language. It is designed to be easy to read and write, making it ideal for beginners and professionals alike. Python code runs line-by-line using an interpreter.

Real-World Analogy
Python is like a recipe book. You write each instruction step by step. The Python interpreter reads and executes each step one at a time — just like a chef following a recipe.

1.1 Print — Hello World!

The print() function displays output to the screen. It is the simplest and most commonly used function in Python.

```
# The print() function outputs text to the screen
# 'Hello, World!' is a string — text enclosed in quotes
print("Hello, World!")   # Output: Hello, World!

# You can print numbers too
print(42)                # Output: 42

# You can print multiple items separated by commas
print("Name:", "Alice", "Age:", 25)  # Output: Name: Alice Age: 25

# sep= changes the separator between items (default is space)
print("A", "B", "C", sep="-")  # Output: A-B-C

# end= changes what is printed at the end (default is newline)
print("Hello", end=" ")  # stays on same line
print("World")           # Output: Hello World
```

1.2 Python as a Calculator

Python can directly evaluate mathematical expressions. You can use it like a powerful calculator.

```
# Basic arithmetic in Python (acts as a calculator)
print(5 + 3)    # Addition        → Output: 8
```

```
print(10 - 4)    # Subtraction      → Output: 6
print(6 * 7)     # Multiplication   → Output: 42
print(15 / 4)    # Division         → Output: 3.75
print(15 // 4)   # Floor Division   → Output: 3   (removes decimal)
print(15 % 4)    # Modulus          → Output: 3   (remainder)
print(2 ** 10)   # Exponent         → Output: 1024
```

1.3 Running Python Code

There are multiple ways to run Python code:

- Interactive Mode: Open terminal and type 'python3' — write and run code line by line.
- Script Mode: Save code in a .py file (e.g., hello.py) and run with: python3 hello.py
- IDE Mode: Use PyCharm, VS Code, or IDLE to write and run full programs.
- Online: Use replit.com or google colab to run Python in the browser.

1.4 Interpreter vs Compiler

Feature	Interpreter (Python)	Compiler (C, Java)
Execution	Line-by-line	Whole program at once
Speed	Slower	Faster
Error detection	Stops at first error	Reports all errors
Output file	No separate file	Generates .exe or bytecode
Examples	Python, Ruby	C, C++, Java

1.5 Python Execution Steps

- Step 1: You write Python code (.py file)
- Step 2: Python compiles it to bytecode (.pyc)
- Step 3: The Python Virtual Machine (PVM) executes the bytecode
- Step 4: Output is shown on screen

Key Insight: Python is both compiled AND interpreted. It compiles to bytecode first, then interprets it. This is why Python balances ease of use with decent performance.

Chapter 2

Variables in Python

Naming, Assignment & Best Practices

2. Variables in Python

What is a Variable?
A variable is a named storage location in memory that holds a value. In Python, you do NOT need to declare a type — Python detects it automatically. Variables are created the moment you assign a value to them.

Real-World Analogy
A variable is like a labelled box. The label is the variable name, and whatever you put inside the box is the value. You can replace the contents anytime.

2.1 Creating & Assigning Variables

```
# Assigning values to variables
name = 'Alice'      # string variable — holds text
age = 20            # integer variable — holds whole number
gpa = 8.7           # float variable — holds decimal number
is_student = True   # boolean variable — holds True or False

# Printing variables
print(name)         # Output: Alice
print(age)          # Output: 20
print(gpa)          # Output: 8.7
print(is_student)   # Output: True

# Python detects type automatically
print(type(name))   # Output: <class 'str'>
print(type(age))    # Output: <class 'int'>
```

2.2 Multiple Assignment

```
# Assign same value to multiple variables at once
x = y = z = 0      # all three variables hold 0

# Assign different values in one line
a, b, c = 10, 20, 30   # a=10, b=20, c=30
print(a, b, c)          # Output: 10 20 30
```

```
# Swap two variables (Python makes this easy!)
a, b = b, a   # swaps without a temp variable
print(a, b)   # Output: 20 10
```

2.3 Variable Naming Rules

Rule	Valid	Invalid
Letters, digits, underscore	student_name, age2	2age, @name
Cannot start with digit	name1, _total	1name
Case-sensitive	Name != name	—
No reserved keywords	my_class, for_loop	class, for, if

```
# Good variable naming (snake_case is standard in Python)
student_name = 'Ravi'     # clear and descriptive
total_marks = 450         # use underscores for spaces
is_passed = True          # booleans often start with 'is_'

# Check all valid variable names with keywords module
import keyword
print(keyword.kwlist)  # prints all Python reserved keywords
```

Chapter 3

Operators in Python

All Types: Arithmetic, Logical, Bitwise & More

3. Operators in Python

What are Operators?
Operators are special symbols that perform operations on values and variables (operands). Python supports 7 types of operators: Arithmetic, Assignment, Comparison, Logical, Bitwise, Identity, and Membership.

3.1 Arithmetic Operators

Operator	Name	Example	Output	Real Use
+	Addition	5+3	8	Total marks
-	Subtraction	9-4	5	Balance
*	Multiplication	6*2	12	Area
/	Division	10/2	5.0	Average
//	Floor Div	7//2	3	Page numbers
%	Modulus	10%3	1	Even/Odd check
**	Exponent	2**3	8	Power/square

```
# Arithmetic operators — real-world example: Shopping bill calculator
price_per_item = 25       # price of one item
quantity = 4              # how many items bought
discount = 15             # discount in rupees

subtotal = price_per_item * quantity   # 25 * 4 = 100
total = subtotal - discount            # 100 - 15 = 85
gst = total * 0.18                     # 18% tax = 15.3
final_bill = total + gst               # 85 + 15.3 = 100.3

print(f'Subtotal: {subtotal}')         # 100
print(f'After discount: {total}')      # 85
print(f'GST (18%): {gst:.2f}')         # 15.30
print(f'Final Bill: {final_bill:.2f}') # 100.30
```

3.2 Assignment Operators

```
# Assignment operators modify and reassign a variable
score = 100             # basic assignment: score = 100
score += 10             # add and assign: score = score + 10 = 110
score -= 5              # subtract and assign: 110 - 5 = 105
score *= 2              # multiply and assign: 105 * 2 = 210
score //= 3             # floor divide and assign: 210 // 3 = 70
score **= 2             # exponent and assign: 70 ** 2 = 4900
score %= 100            # modulus and assign: 4900 % 100 = 0
print(score)            # Output: 0
```

3.3 Comparison (Relational) Operators

Operator	Meaning	Example	Output
==	Equal to	5==5	True
!=	Not equal	5!=3	True
>	Greater than	7>3	True
<	Less than	2<4	True
>=	Greater or equal	6>=6	True
<=	Less or equal	4<=5	True

3.4 Logical Operators

```
# Logical operators combine conditions
# and → True only if BOTH conditions are True
age = 20
has_id = True
can_enter = (age >= 18) and has_id    # True and True = True
print('Can enter:', can_enter)        # True

# or → True if AT LEAST ONE condition is True
is_weekend = False
is_holiday = True
can_rest = is_weekend or is_holiday   # False or True = True
print('Can rest:', can_rest)          # True

# not → reverses the result
is_raining = False
go_outside = not is_raining           # not False = True
print('Go outside:', go_outside)      # True
```

3.5 Bitwise Operators

Bitwise operators work on binary (0s and 1s) representations of numbers. Used in low-level programming and optimization.

```
# Bitwise operators — work on binary bits
a = 5   # binary: 0101
b = 3   # binary: 0011

print(a & b)   # AND: 0101 & 0011 = 0001 → 1
print(a | b)   # OR:  0101 | 0011 = 0111 → 7
print(a ^ b)   # XOR: 0101 ^ 0011 = 0110 → 6
print(~a)      # NOT: ~0101 = -(5+1)     → -6
print(a << 1)  # Left shift: 0101 → 1010  → 10 (multiply by 2)
print(a >> 1)  # Right shift: 0101 → 0010 → 2  (divide by 2)
```

3.6 Identity & Membership Operators

```
# Identity operators — compare memory location, not just value
a = [1, 2, 3]
b = a          # b points to SAME list as a
c = [1, 2, 3]  # c is a NEW list with same values

print(a is b)       # True  — same object in memory
print(a is c)       # False — different objects
print(a is not c)   # True

# Membership operators — check if value is IN a collection
fruits = ['apple', 'banana', 'cherry']
print('apple' in fruits)      # True  — apple is in the list
print('mango' in fruits)      # False — mango is not in the list
print('mango' not in fruits)  # True

# Works on strings too
email = 'student@college.edu'
print('@' in email)           # True  — valid email check
```

Chapter 4

Keywords & Data Types

Reserved Words and Python's Type System

4. Keywords in Python

What are Keywords?
Keywords are reserved words in Python that have a special pre-defined meaning. They cannot be used as variable names, function names, or identifiers. Python has 35+ keywords.

```
# To see all Python keywords:
import keyword
print(keyword.kwlist)
# Output: ['False', 'None', 'True', 'and', 'as', 'assert',
#          'async', 'await', 'break', 'class', 'continue', 'def',
#          'del', 'elif', 'else', 'except', 'finally', 'for',
#          'from', 'global', 'if', 'import', 'in', 'is', 'lambda',
#          'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
#          'try', 'while', 'with', 'yield']

# Using a keyword as a variable name causes SyntaxError:
# for = 10    # ERROR: SyntaxError: invalid syntax
```

Category	Keywords
Values	True, False, None
Operators	and, or, not, is, in
Control Flow	if, else, elif, for, while, break, continue, pass
Error Handling	try, except, finally, raise, assert
Functions/Classes	def, return, lambda, yield, class
Context	with, as
Import/Scope	import, from, global, nonlocal
Async	async, await

5. Data Types in Python

What are Data Types?
Data types define the kind of value a variable holds. Python is dynamically typed — the interpreter automatically determines the type. In Python, every value is an object, and every

data type is a class.

5.1 Numeric Types

```
# 1. int — whole numbers (positive or negative)
population = 1_400_000_000   # can use _ for readability
temperature = -15            # negative integer
print(type(population))      # <class 'int'>

# 2. float — decimal numbers
pi = 3.14159
price = 99.99
print(type(pi))              # <class 'float'>

# 3. complex — numbers with real + imaginary parts
signal = 3 + 4j              # used in engineering/physics
print(signal.real)           # 3.0 — real part
print(signal.imag)           # 4.0 — imaginary part
print(type(signal))          # <class 'complex'>
```

5.2 String Type

```
# str — sequence of characters
name = 'Priya'
city = "Chennai"
bio = '''I am a Python student
learning programming in 2025'''  # multi-line string

print(type(name))    # <class 'str'>
print(len(name))     # 5 — number of characters
print(name[0])       # P — first character
print(name[-1])      # a — last character
```

5.3 Boolean Type

```
# bool — only two values: True or False
is_logged_in = True
is_admin = False

# In Python, booleans are subclass of int:
print(int(True))   # 1
print(int(False))  # 0
print(True + True) # 2

# bool() converts values to True/False
print(bool(0))     # False — zero is falsy
```

```
print(bool(''))    # False — empty string is falsy
print(bool(42))    # True  — any non-zero is truthy
print(bool('Hi'))  # True  — non-empty string is truthy
```

5.4 Type Casting (Type Conversion)

Type Casting

Type casting is converting a value from one data type to another. There are two types: Implicit (automatic, by Python) and Explicit (manual, by programmer).

```
# IMPLICIT type casting — Python does it automatically
x = 5        # int
y = 2.5      # float
result = x + y   # Python auto-converts int to float
print(result)    # 7.5
print(type(result))  # <class 'float'>

# EXPLICIT type casting — programmer does it manually
# int() — convert to integer
price = int('250')   # string '250' → integer 250
score = int(98.7)    # float 98.7 → integer 98 (truncates)

# float() — convert to float
pi_approx = float('3.14')   # string → float
ratio = float(3)            # int → float: 3.0

# str() — convert to string
age = 20
message = 'I am ' + str(age) + ' years old'
print(message)   # I am 20 years old

# list(), tuple(), set() — convert collections
my_string = 'hello'
letters = list(my_string)   # ['h','e','l','l','o']
unique = set(my_string)     # {'h','e','l','o'} — removes duplicates
```

5.5 Input Function

input() Function

The input() function reads user input from the keyboard as a string. You must cast (convert) it to the required type.

```
# input() — reads keyboard input, always returns a string
name = input('Enter your name: ')       # gets string input
age = int(input('Enter your age: '))    # cast to int
```

```
gpa = float(input('Enter your GPA: '))  # cast to float

# Real-world example: Simple student info system
print(f'Hello {name}!')
if age >= 18:
    print('You are an adult student.')
print(f'Your GPA is {gpa:.2f}')

# Safe input with try-except
try:
    marks = int(input('Enter marks (0-100): '))
    print(f'You scored {marks} out of 100')
except ValueError:
    print('Invalid input! Please enter a number.')
```

Chapter 5

Control Flow Statements

if/elif/else, match-case

6. Conditional Statements

What are Conditional Statements?
Conditional statements let your program make decisions. Code runs ONLY if a certain condition is True. Python uses if, elif, and else keywords for this.

Real-World Analogy
Think of conditional statements like traffic lights: IF green → go, ELIF yellow → slow down, ELSE (red) → stop.

6.1 if Statement

```
# if statement — runs code only if condition is True
marks = 75

if marks >= 60:                      # condition to check
    print('You passed the exam!')    # runs if condition is True

# Note: Python uses INDENTATION (4 spaces) — not braces {}
```

6.2 if-else Statement

```
# if-else — two-way decision
temperature = 35

if temperature > 30:
    print('It is hot outside.')      # runs if True
else:
    print('It is cool outside.')     # runs if False
```

6.3 if-elif-else (Multiple Conditions)

```
# if-elif-else — multi-way decision (like a grade calculator)
marks = 82
```

```
if marks >= 90:
    grade = 'A+'        # 90 and above
elif marks >= 80:
    grade = 'A'         # 80 to 89
elif marks >= 70:
    grade = 'B'         # 70 to 79
elif marks >= 60:
    grade = 'C'         # 60 to 69
elif marks >= 50:
    grade = 'D'         # 50 to 59
else:
    grade = 'F'         # below 50 — fail

print(f'Marks: {marks} → Grade: {grade}')  # Marks: 82 → Grade: A
```

6.4 Nested if Statements

```
# Nested if — if inside another if
# Real-world: ATM machine logic
balance = 5000
pin_correct = True
amount = 2000

if pin_correct:                         # outer: check PIN first
    if amount <= balance:               # inner: check sufficient balance
        balance -= amount
        print(f'Dispensing Rs.{amount}. Remaining: Rs.{balance}')
    else:
        print('Insufficient balance!')
else:
    print('Wrong PIN! Card blocked.')
```

6.5 Ternary (One-line if-else)

```
# Ternary expression — short form of if-else in one line
# Syntax: value_if_true if condition else value_if_false
age = 20
status = 'Adult' if age >= 18 else 'Minor'
print(status)  # Adult

# Another example: pass/fail
marks = 45
result = 'Pass' if marks >= 35 else 'Fail'
print(result)  # Pass
```

6.6 Match-Case Statement (Python 3.10+)

match-case

match-case is Python's version of switch-case from other languages. It matches a variable's value against a set of patterns and runs the matching block.

```
# match-case — like a menu selector
# Real-world: Weekday name from number
day = 3

match day:
    case 1:
        print('Monday')
    case 2:
        print('Tuesday')
    case 3:
        print('Wednesday')    # ← this matches!
    case 4 | 5:                # | means OR — matches 4 or 5
        print('Thu or Fri')
    case _:                    # _ is default (like else)
        print('Weekend')

# Real-world: HTTP status code handler
status_code = 404
match status_code:
    case 200: print('OK - Success')
    case 404: print('Not Found')
    case 500: print('Server Error')
    case _:   print('Unknown status')
```

Chapter 6

Loops in Python

while, for, range, nested loops & control statements

7. Loops in Python

What are Loops?
Loops execute a block of code repeatedly. Instead of writing the same code many times, you use a loop. Python has two main loop types: while (condition-based) and for (collection-based).

Real-World Analogy
A loop is like an alarm clock — it keeps repeating an action (beeping) until a condition is met (you wake up and turn it off).

7.1 While Loop

```
# while loop — runs AS LONG AS the condition is True
# Real-world: ATM PIN entry (max 3 attempts)
correct_pin = '1234'
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    entered_pin = input('Enter PIN: ')
    attempts += 1   # increment attempt counter
    if entered_pin == correct_pin:
        print('Access Granted!')
        break        # exit loop immediately
    else:
        remaining = max_attempts - attempts
        print(f'Wrong PIN! {remaining} attempts left.')
else:
    # else block runs only if loop completed WITHOUT break
    print('Card blocked after 3 failed attempts!')
```

7.2 For Loop

```
# for loop — iterates over a sequence (list, string, range, etc.)

# Iterating over a list
students = ['Alice', 'Bob', 'Charlie', 'Diana']
```

```
for student in students:        # student gets each value one by one
    print(f'Hello, {student}!')

# Iterating over a string
word = 'Python'
for char in word:               # char gets each letter
    print(char, end=' ')        # Output: P y t h o n

# Iterating over a dictionary
marks = {'Math': 90, 'Science': 85, 'English': 78}
for subject, score in marks.items():
    print(f'{subject}: {score}')
```

7.3 Range Function

```
# range() generates a sequence of numbers
# range(stop)          → 0 to stop-1
# range(start, stop)   → start to stop-1
# range(start, stop, step) → with custom step

# Count from 0 to 4
for i in range(5):           # 0,1,2,3,4
    print(i, end=' ')
print()   # newline

# Count from 1 to 10
for i in range(1, 11):       # 1,2,...,10
    print(i, end=' ')
print()

# Count by 2s (even numbers)
for i in range(0, 20, 2):    # 0,2,4,...,18
    print(i, end=' ')
print()

# Count backwards
for i in range(10, 0, -1):   # 10,9,8,...,1
    print(i, end=' ')
```

7.4 Loop Control Statements

```
# break — exits the loop immediately
# Real-world: Search for a product in inventory
inventory = ['pen', 'book', 'ruler', 'eraser', 'pencil']
search = 'ruler'
for item in inventory:
    if item == search:
        print(f'Found: {item}!')
```

```
        break               # stop searching once found
    print(f'Checking {item}...')

# continue — skips current iteration, goes to next
# Real-world: Skip failed students in result list
marks_list = [85, 92, 45, 78, 30, 95]
print('Passed students marks:')
for m in marks_list:
    if m < 50:              # skip failed marks
        continue
    print(m)

# pass — does nothing (placeholder for empty blocks)
for i in range(5):
    if i == 3:
        pass                # to be implemented later
    else:
        print(i)
```

7.5 Nested Loops

```
# Nested loop — loop inside another loop
# Real-world: Multiplication table
for i in range(1, 4):          # outer loop: rows 1 to 3
    for j in range(1, 4):      # inner loop: columns 1 to 3
        print(f'{i}x{j}={i*j}', end='  ')
    print()  # newline after each row

# Output:
# 1x1=1  1x2=2  1x3=3
# 2x1=2  2x2=4  2x3=6
# 3x1=3  3x2=6  3x3=9

# Real-world: Print a star pattern
rows = 5
for i in range(1, rows + 1):
    print('* ' * i)
```

Chapter 7

Functions in Python

Built-in, User-defined, Lambda, Recursion & More

8. Functions in Python

What is a Function?
A function is a reusable block of code that performs a specific task. You define it once and call it whenever needed. Functions make code organized, readable, and maintainable.

Real-World Analogy
A function is like a vending machine: you put in an input (press a button), it processes (internal mechanism), and returns an output (gives you a snack). You don't need to know HOW it works inside — just what goes in and what comes out.

8.1 Defining & Calling a Function

```
# Define a function using 'def' keyword
def greet_student(name):        # name is a parameter (input)
    '''This function greets a student by name.'''  # docstring
    message = f'Hello, {name}! Welcome to Python class.'
    return message              # return sends value back

# Call the function
result = greet_student('Ravi')  # 'Ravi' is the argument
print(result)                   # Hello, Ravi! Welcome to Python class.

# Access docstring
print(greet_student.__doc__)    # This function greets a student by name.
```

8.2 Types of Function Arguments

Positional Arguments

```
# Positional arguments — ORDER matters
def register(name, age, course):
    print(f'{name} (age {age}) enrolled in {course}')

register('Alice', 20, 'Python')    # correct order
# register(20, 'Alice', 'Python')  # wrong — age in name's position
```

Keyword Arguments

```
# Keyword arguments — use parameter name, order does NOT matter
def book_ticket(passenger, seat, class_type):
    print(f'{passenger} → Seat {seat} ({class_type})')

book_ticket(seat='12A', class_type='Business', passenger='Bob')
# Output: Bob → Seat 12A (Business)
```

Default Arguments

```
# Default arguments — provide fallback value if not given
def calculate_interest(principal, rate=7.5, years=1):
    interest = (principal * rate * years) / 100
    return interest

print(calculate_interest(10000))           # uses default rate=7.5,
years=1
print(calculate_interest(10000, 8.5, 3))   # overrides defaults
```

Variable Arguments (*args and **kwargs)

```
# *args — accepts any number of positional arguments as a tuple
def total_marks(*subjects):     # * collects all args into tuple
    return sum(subjects)

print(total_marks(85, 90, 78))        # 3 subjects
print(total_marks(85, 90, 78, 92, 88))  # 5 subjects

# **kwargs — accepts any number of keyword arguments as a dict
def student_info(**details):    # ** collects all keyword args into dict
    for key, value in details.items():
        print(f'{key}: {value}')

student_info(name='Alice', age=20, course='Python', batch='2025')
```

8.3 Return Statement

```
# return sends a value back to the caller
def bmi_calculator(weight_kg, height_m):
    '''Calculate Body Mass Index.'''
    bmi = weight_kg / (height_m ** 2)   # BMI formula
    if bmi < 18.5:
        category = 'Underweight'
    elif bmi < 25:
        category = 'Normal'
    elif bmi < 30:
```

```
        category = 'Overweight'
    else:
        category = 'Obese'
    return bmi, category   # return MULTIPLE values as a tuple

bmi, status = bmi_calculator(70, 1.75)
print(f'BMI: {bmi:.2f} — {status}')  # BMI: 22.86 — Normal
```

8.4 Lambda Functions

Lambda Functions

A lambda function is a small, anonymous (nameless) one-line function. It uses the lambda keyword instead of def. Best for short, simple functions used temporarily.

```
# Lambda syntax: lambda arguments: expression

# Regular function
def square(x):
    return x ** 2

# Same function as lambda
square_lambda = lambda x: x ** 2
print(square_lambda(5))   # 25

# Lambda with conditions
result = lambda marks: 'Pass' if marks >= 35 else 'Fail'
print(result(40))  # Pass
print(result(28))  # Fail

# Lambda with multiple arguments
area = lambda l, w: l * w
print(area(5, 3))  # 15

# Lambda with map() — apply to each element
prices = [100, 200, 300, 400]
discounted = list(map(lambda p: p * 0.9, prices))  # 10% off
print(discounted)  # [90.0, 180.0, 270.0, 360.0]

# Lambda with filter() — keep elements matching condition
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda n: n % 2 == 0, numbers))
print(evens)  # [2, 4, 6, 8, 10]

# Lambda with sorted() — sort students by marks
students =
[{'name':'Alice','marks':85},{'name':'Bob','marks':75},{'name':'Charlie','marks':90}]
ranked = sorted(students, key=lambda s: s['marks'], reverse=True)
for s in ranked:
    print(f"{s['name']}: {s['marks']}")
```

8.5 Nested Functions & Closures

```
# Nested function — function defined inside another function
def outer_function(message):
    def inner_function():          # inner function has access to outer's
variables
        print(f'Message: {message}')
    inner_function()               # call inner from outer

outer_function('Hello from nested!')  # Message: Hello from nested!

# Closure — inner function REMEMBERS outer variable even after outer
finishes
def create_multiplier(factor):     # factory function
    def multiply(number):          # closure — remembers 'factor'
        return number * factor
    return multiply                # return the inner function

double = create_multiplier(2)      # factor=2 is remembered
triple = create_multiplier(3)      # factor=3 is remembered

print(double(5))   # 10 — 5 * 2
print(triple(5))   # 15 — 5 * 3
```

8.6 Decorators

Decorators
A decorator is a function that wraps another function to add extra behaviour without changing its original code. Uses the @symbol above the function definition.

```
# Decorator — adds functionality before/after a function

def timing_decorator(func):          # wrapper function
    import time
    def wrapper(*args, **kwargs):    # wrapper replaces the original
        start = time.time()          # record start time
        result = func(*args, **kwargs)  # call original function
        end = time.time()            # record end time
        print(f'{func.__name__} took {end-start:.4f} seconds')
        return result
    return wrapper

@timing_decorator              # apply decorator with @ symbol
def slow_sum(n):
    return sum(range(n))

total = slow_sum(1000000)      # slow_sum took 0.0312 seconds
```

```
print(total)

# Common use cases for decorators:
# @login_required  — check if user is logged in before running
# @cache           — remember results to avoid recalculation
# @retry           — retry function on failure
```

Chapter 8

Recursion in Python

Self-calling functions, base cases, types

9. Recursion

What is Recursion?
Recursion is a technique where a function calls itself to solve smaller sub-problems of the same type. Every recursive function must have a BASE CASE (stopping condition) to prevent infinite loops.

Real-World Analogy
Think of Russian nesting dolls (Matryoshka). To find the smallest doll, you open the biggest, which contains a smaller one, which contains an even smaller one... until you reach the tiniest doll. That tiny doll is the base case.

9.1 Structure of Recursive Function

```
# Every recursive function has TWO parts:
def recursive_function(n):
    # 1. BASE CASE — stops the recursion
    if n <= 0:
        return 0
    # 2. RECURSIVE CASE — calls itself with smaller problem
    return n + recursive_function(n - 1)

print(recursive_function(5))  # 5+4+3+2+1+0 = 15
```

9.2 Factorial

```
# Factorial: n! = n × (n-1) × ... × 1
# Example: 5! = 5×4×3×2×1 = 120

def factorial(n):
    if n == 0 or n == 1:    # base case: 0! = 1! = 1
        return 1
    return n * factorial(n - 1)  # recursive case

# How it works for factorial(4):
# factorial(4) → 4 * factorial(3)
# factorial(3) → 3 * factorial(2)
```

```
# factorial(2) → 2 * factorial(1)
# factorial(1) → 1  [BASE CASE]
# Result: 2*1=2, 3*2=6, 4*6=24

print(factorial(5))   # 120
print(factorial(0))   # 1
print(factorial(10))  # 3628800
```

9.3 Fibonacci Sequence

```
# Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21...
# Each number = sum of previous two

def fibonacci(n):
    if n == 0: return 0    # base case 1
    if n == 1: return 1    # base case 2
    return fibonacci(n-1) + fibonacci(n-2)  # recursive

# Print first 10 Fibonacci numbers
for i in range(10):
    print(fibonacci(i), end=' ')
# Output: 0 1 1 2 3 5 8 13 21 34
```

9.4 Tail vs Non-Tail Recursion

Type	Definition	Example
Tail Recursion	Recursive call is the LAST operation	tail_factorial(n, acc=1)
Non-Tail Recursion	Operations happen AFTER recursive call	n * factorial(n-1)

9.5 Recursion vs Iteration

Feature	Recursion	Iteration
Method	Function calls itself	Uses for/while loops
Memory	More (call stack grows)	Less (reuses variables)
Readability	Cleaner for tree problems	Better for linear problems
Speed	Slower (function call overhead)	Faster
Best for	Trees, backtracking, D&C	Simple repetition

Chapter 9

Strings in Python

Methods, Slicing, Formatting & Real-World Use

10. Strings in Python

What is a String?
A string is a sequence of characters enclosed in single, double, or triple quotes. Strings are immutable — you cannot change individual characters after creation. Every character has an index position.

Real-World Analogy
A string is like a row of seats in a cinema. Each seat (character) has a number (index) starting from 0. You can look at any seat by its number, but you cannot swap seats without creating a new row arrangement.

10.1 Creating Strings

```
# Different ways to create strings
s1 = 'Hello'                    # single quotes
s2 = "World"                    # double quotes
s3 = '''Multi
line string'''                  # triple quotes — spans multiple lines
s4 = "It's a great day!"        # use double when string has apostrophe

# f-string (formatted string) — embed variables directly
name, age = 'Alice', 20
intro = f'My name is {name} and I am {age} years old.'
print(intro)    # My name is Alice and I am 20 years old.

# format() method
msg = 'Hello, {}! You scored {}/100'.format('Bob', 95)
print(msg)      # Hello, Bob! You scored 95/100
```

10.2 String Indexing & Slicing

```
# String indexing — access individual characters
# Index:  0  1  2  3  4  5
# Char:   P  y  t  h  o  n
# Neg:   -6 -5 -4 -3 -2 -1
word = 'Python'
print(word[0])    # P  — first character
print(word[-1])   # n  — last character
```

```
print(word[2])    # t

# String slicing — extract a portion
# Syntax: string[start:stop:step]
text = 'Programming'
print(text[0:4])     # Prog  — index 0,1,2,3
print(text[4:])      # ramming — from index 4 to end
print(text[:4])      # Prog  — from start to index 3
print(text[::2])     # Pormig — every 2nd character
print(text[::-1])    # gnimmargorP — REVERSE a string!
```

10.3 Key String Methods

```
s = '  Hello, World!  '

print(s.strip())          # 'Hello, World!'  — remove spaces
print(s.lower())          # '  hello, world!  '
print(s.upper())          # '  HELLO, WORLD!  '
print(s.title())          # '  Hello, World!  '
print(s.replace('World','Python'))  # '  Hello, Python!  '
print(s.strip().split(', '))  # ['Hello', 'World!']
print(s.strip().startswith('H'))  # True
print(s.strip().endswith('!'))    # True
print(s.strip().count('l'))       # 3 — count occurrences
print(s.strip().find('World'))    # 7 — index of first occurrence

# join — combine list of strings
words = ['Python', 'is', 'awesome']
sentence = ' '.join(words)       # join with space
print(sentence)                  # Python is awesome

# check content
print('123'.isdigit())    # True  — all digits
print('abc'.isalpha())    # True  — all letters
print('abc123'.isalnum()) # True  — alphanumeric
```

10.4 Escape Characters

Escape	Meaning	Example	Output
\n	Newline	'Hello\nWorld'	Hello (newline) World
\t	Tab	'A\tB'	A B
\\'	Single Quote	'It\\'s cool'	It's cool
\\"	Double Quote	"He said \\"Hi\\""	He said "Hi"
\\\\	Backslash	'C:\\\\Users'	C:\Users

10.5 Real-World String Example

```
# Real-world: Email validation and formatting
email = '  Student@College.EDU  '

# Clean and normalize the email
cleaned = email.strip().lower()         # 'student@college.edu'
is_valid = '@' in cleaned and '.' in cleaned

if is_valid:
    username, domain = cleaned.split('@')  # split at @
    print(f'Username: {username}')         # student
    print(f'Domain: {domain}')             # college.edu
    print('Valid email!')
else:
    print('Invalid email format!')
```

Chapter 10

Lists in Python

Create, Access, Modify, Comprehension & Methods

11. Lists in Python

What is a List?
A list is an ordered, mutable (changeable) collection of items. Lists can hold different data types. They are created with square brackets [] and are one of Python's most commonly used data structures.

Real-World Analogy
A list is like a shopping cart — you can add items, remove items, rearrange them, and look at any item by its position number.

11.1 Creating Lists

```
# Creating lists
empty_list = []                          # empty list
numbers = [1, 2, 3, 4, 5]               # list of integers
mixed = [42, 'hello', 3.14, True]        # mixed types are allowed
nested = [[1,2,3], [4,5,6]]             # list of lists (matrix)

# list() constructor
letters = list('Python')                # ['P','y','t','h','o','n']
repeated = [0] * 5                      # [0, 0, 0, 0, 0]

print(numbers[0])    # 1 — first element
print(numbers[-1])   # 5 — last element
print(len(numbers)) # 5 — length of list
```

11.2 List Methods

```
fruits = ['banana', 'apple', 'cherry']

fruits.append('mango')        # add to end
print(fruits)  # ['banana', 'apple', 'cherry', 'mango']

fruits.insert(1, 'grape')     # insert at index 1
print(fruits)  # ['banana', 'grape', 'apple', 'cherry', 'mango']
```

```
fruits.remove('apple')        # remove by value
popped = fruits.pop(0)        # remove by index → returns 'banana'
del fruits[0]                 # delete by index

nums = [3,1,4,1,5,9,2,6]
nums.sort()                   # sort ascending in-place
print(nums)    # [1, 1, 2, 3, 4, 5, 6, 9]
nums.sort(reverse=True)       # sort descending
nums.reverse()                # reverse in-place
print(nums.count(1))          # 2 — count occurrences
print(nums.index(5))          # position of first 5
copy = nums.copy()            # shallow copy
nums.clear()                  # remove all elements
```

11.3 List Slicing & Iteration

```
marks = [85, 92, 78, 95, 88, 73, 90]

print(marks[2:5])    # [78, 95, 88] — index 2,3,4
print(marks[:3])     # [85, 92, 78] — first 3
print(marks[-3:])    # [73, 90, ...] — last 3
print(marks[::2])    # every 2nd element

# Iterating with index using enumerate()
for i, mark in enumerate(marks):
    print(f'Student {i+1}: {mark}')
```

11.4 List Comprehension

List Comprehension
A concise one-line way to create a new list by applying an expression to each element of an iterable, optionally filtering with a condition.

```
# Syntax: [expression for item in iterable if condition]

# Squares of 1 to 10
squares = [x**2 for x in range(1, 11)]
print(squares)  # [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

# Only even numbers from 1-20
evens = [x for x in range(1, 21) if x % 2 == 0]
print(evens)    # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

# Transform: uppercase all names
names = ['alice', 'bob', 'charlie']
upper_names = [n.upper() for n in names]
print(upper_names)  # ['ALICE', 'BOB', 'CHARLIE']
```

```
# Filter + transform: marks of students who passed (>= 60)
all_marks = [85, 45, 92, 38, 76, 55, 88]
passed = [m for m in all_marks if m >= 60]
print(passed)   # [85, 92, 76, 88]

# 2D list comprehension: multiplication table
mult_table = [[i*j for j in range(1, 4)] for i in range(1, 4)]
for row in mult_table:
    print(row)
# [1, 2, 3]
# [2, 4, 6]
# [3, 6, 9]
```

Chapter 11

Tuples in Python

Immutable Sequences, Unpacking & Operations

12. Tuples in Python

What is a Tuple?
A tuple is an ordered, immutable (cannot be changed) collection of items. Created with parentheses (). Tuples are faster than lists and are used for fixed data that should not be modified.

Real-World Analogy
A tuple is like a birth certificate — it records fixed, unchangeable facts (name, date, place of birth). Unlike a list (shopping cart) that changes, a tuple's contents stay the same forever.

12.1 Creating Tuples

```
# Creating tuples
empty = ()                          # empty tuple
single = (42,)                      # single element — note the COMMA
coords = (10.5, 20.3)               # coordinates
rgb = (255, 128, 0)                 # color values
mixed = ('Alice', 20, True, 3.14)   # mixed types

# Without parentheses (tuple packing)
point = 3, 4, 5                     # creates a tuple (3, 4, 5)

# From list or string
t1 = tuple([1, 2, 3])      # (1, 2, 3)
t2 = tuple('Hello')        # ('H', 'e', 'l', 'l', 'o')
```

12.2 Tuple Operations

```
t = (10, 20, 30, 40, 50)

# Accessing
print(t[0])        # 10
print(t[-1])       # 50
print(t[1:4])      # (20, 30, 40)

# Tuple methods
```

```
print(t.count(20))  # 1 — count occurrences
print(t.index(30))  # 2 — position of 30

# Concatenation
t2 = (60, 70)
combined = t + t2  # (10,20,30,40,50,60,70)

# Repetition
repeated = (1, 2) * 3  # (1, 2, 1, 2, 1, 2)
```

12.3 Tuple Unpacking

```
# Basic unpacking — assign tuple elements to variables
person = ('Alice', 20, 'Python Developer')
name, age, job = person   # unpack into 3 variables
print(name, age, job)     # Alice 20 Python Developer

# Star operator (*) — capture remaining items
scores = (100, 95, 88, 72, 65, 50)
first, second, *rest = scores
print(first)   # 100
print(second)  # 95
print(rest)    # [88, 72, 65, 50]

# Swap variables using tuple packing/unpacking
x, y = 10, 20
x, y = y, x      # Python magic — no temp variable needed
print(x, y)      # 20 10
```

12.4 List vs Tuple

Feature	List	Tuple
Syntax	[ ]	( )
Mutability	Mutable (can change)	Immutable (cannot change)
Performance	Slower	Faster (for reading)
Methods	Many (append, remove, etc.)	Only count() and index()
Use Case	Dynamic data	Fixed data, DB records, coordinates
Memory	More	Less

Chapter 12

Dictionaries in Python

Key-Value Pairs, Methods & Comprehension

13. Dictionaries in Python

What is a Dictionary?
A dictionary is an unordered (Python 3.7+ maintains insertion order), mutable collection of key-value pairs. Created with curly braces {}. Keys must be unique and immutable.

Real-World Analogy
A dictionary is like a real dictionary or a contact book: you look up a word (key) to find its definition (value). You don't search page by page — you go directly to the key.

13.1 Creating Dictionaries

```
# Creating dictionaries
empty = {}                                  # empty dict
student = {'name': 'Alice', 'age': 20}      # string keys
product = {'id': 101, 'price': 599.99}      # mixed value types

# dict() constructor
config = dict(host='localhost', port=3306, db='school')

# Accessing values
print(student['name'])             # Alice — via key
print(student.get('age'))          # 20 — safer (no KeyError)
print(student.get('grade', 'N/A')) # N/A — default if key missing
```

13.2 CRUD Operations

```
# CREATE — add new key-value pair
student = {'name': 'Bob', 'age': 21}
student['course'] = 'Python'       # add new key
student.update({'year': 2, 'gpa': 8.5})  # add multiple

# READ — access values
print(student.keys())    # dict_keys(['name','age','course','year','gpa'])
print(student.values())  # dict_values(['Bob', 21, 'Python', 2, 8.5])
print(student.items())   # dict_items([('name','Bob'), ...])
```

```
# UPDATE — modify existing value
student['age'] = 22      # direct assignment
student['gpa'] = 9.0     # update gpa

# DELETE
del student['year']      # remove by key
removed = student.pop('course')  # remove & return value
student.clear()          # remove all
```

13.3 Iterating Dictionaries

```
marks = {'Math': 90, 'Science': 85, 'English': 78, 'History': 82}

# Iterate over keys (default)
for subject in marks:
    print(subject)           # Math, Science, English, History

# Iterate over values
for score in marks.values():
    print(score)             # 90, 85, 78, 82

# Iterate over key-value pairs
for subject, score in marks.items():
    grade = 'A' if score >= 85 else 'B' if score >= 75 else 'C'
    print(f'{subject}: {score} ({grade})')
```

13.4 Nested Dictionary

```
# Nested dictionary — dict inside dict
# Real-world: Student database
students = {
    'S001': {'name': 'Alice', 'marks': {'Math':90,'Sci':88}},
    'S002': {'name': 'Bob',   'marks': {'Math':75,'Sci':82}},
}

# Access nested values
print(students['S001']['name'])            # Alice
print(students['S001']['marks']['Math'])   # 90

# Calculate average for each student
for sid, info in students.items():
    avg = sum(info['marks'].values()) / len(info['marks'])
    print(f'{info["name"]} average: {avg:.1f}')
```

13.5 Dictionary Comprehension

```
# Dict comprehension — create dict in one line
# {key_expr: value_expr for item in iterable if condition}

# Squares of 1-5
squares = {x: x**2 for x in range(1, 6)}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Filter: keep only passed students
marks = {'Alice': 88, 'Bob': 45, 'Charlie': 92, 'Dave': 38}
passed = {name: m for name, m in marks.items() if m >= 50}
print(passed)   # {'Alice': 88, 'Charlie': 92}
```

Chapter 13

Sets in Python

Unique Elements, Set Operations & Methods

14. Sets in Python

What is a Set?
A set is an unordered collection of unique elements. Duplicates are automatically removed. Sets support mathematical operations like union, intersection, and difference.

Real-World Analogy
A set is like a bag of unique coins — no two coins are the same. If you try to add a duplicate coin, it's simply ignored. The bag has no fixed order.

14.1 Creating Sets

```
# Creating sets — duplicates are removed automatically
fruits = {'apple', 'banana', 'cherry', 'apple'}  # duplicate removed
print(fruits)   # {'apple', 'banana', 'cherry'}

# From list — removes duplicates
numbers = [1, 2, 2, 3, 3, 3, 4]
unique = set(numbers)
print(unique)   # {1, 2, 3, 4}

# Empty set — MUST use set(), not {} (that creates empty dict)
empty = set()

# frozenset — immutable version of set
fixed = frozenset([1, 2, 3, 4])
# fixed.add(5)  # ERROR — cannot modify frozenset
```

14.2 Set Operations

```
# Mathematical set operations
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# Union — all elements from both sets
print(A | B)           # {1, 2, 3, 4, 5, 6, 7, 8}
print(A.union(B))      # same result
```

```
# Intersection — only common elements
print(A & B)           # {4, 5}
print(A.intersection(B)) # same result

# Difference — elements in A but NOT in B
print(A - B)           # {1, 2, 3}
print(A.difference(B)) # same result

# Symmetric difference — elements in EITHER but NOT BOTH
print(A ^ B)                  # {1, 2, 3, 6, 7, 8}
print(A.symmetric_difference(B)) # same

# Subset and Superset
C = {1, 2}
print(C.issubset(A))    # True — C ⊆ A
print(A.issuperset(C))  # True — A ⊇ C
```

14.3 Set Methods

```
s = {1, 2, 3}

s.add(4)               # add single element: {1,2,3,4}
s.update([5, 6, 7])    # add multiple: {1,2,3,4,5,6,7}
s.remove(3)            # remove 3 (KeyError if not found)
s.discard(99)          # remove 99 (no error if not found)
popped = s.pop()       # remove & return arbitrary element
s.clear()              # empty the set

# Real-world: Find common interests between two users
user1_interests = {'Python', 'Music', 'Travel', 'Gaming'}
user2_interests = {'Java', 'Music', 'Reading', 'Gaming', 'Cooking'}

common = user1_interests & user2_interests
print('Common interests:', common)   # {'Music', 'Gaming'}

only_user1 = user1_interests - user2_interests
print('Only User1 likes:', only_user1)  # {'Python', 'Travel'}
```

Chapter 14

Arrays in Python

Python Array Module & Comparison with Lists

15. Arrays in Python

What is an Array?
An array stores multiple elements of the SAME data type in contiguous memory. Unlike lists (which can hold mixed types), arrays are memory-efficient for large numerical data.

15.1 Array vs List

Feature	Python List	Python Array	NumPy Array
Types	Mixed types allowed	Same type only	Same type (numerical)
Memory	Less efficient	More efficient	Highly efficient
Speed	General purpose	Good for numeric	Fast (vectorized ops)
Import	Built-in	import array	import numpy as np

15.2 Creating and Using Arrays

```
import array as arr

# Create array — need typecode: 'i'=int, 'f'=float, 'd'=double
temperatures = arr.array('f', [36.5, 37.2, 36.8, 38.1, 37.5])
scores = arr.array('i', [85, 92, 78, 95, 88])

# Access elements
print(temperatures[0])    # 36.5
print(scores[-1])         # 88

# Add elements
temperatures.append(37.0)           # add at end
temperatures.insert(2, 36.9)        # insert at index 2

# Remove elements
scores.remove(78)                   # remove by value
popped = scores.pop(0)              # remove by index

# Slicing
print(temperatures[1:4])            # elements 1, 2, 3
```

```
# Other operations
print(scores.index(92))             # position of 92
print(scores.count(85))             # how many times 85 appears
scores.reverse()                    # reverse in-place
scores.extend([91, 87, 93])         # add multiple elements

# Convert to list
scores_list = scores.tolist()
```

Chapter 15

Stack & Queue in Python

LIFO & FIFO Data Structures

16. Stack & Queue

16.1 Stack — Last In, First Out (LIFO)

What is a Stack?
A stack works like a pile of plates: you always add (push) and remove (pop) from the TOP. The last item added is the first to be removed (LIFO).

Real-World Analogy
Think of a stack of trays in a cafeteria. You place a tray on top, and you also pick from the top. The tray placed last is the first to be picked.

```
# Stack implementation using Python list
stack = []

# PUSH — add items to top
stack.append('Page 1')    # visit page 1
stack.append('Page 2')    # visit page 2
stack.append('Page 3')    # visit page 3
print('Stack:', stack)    # ['Page 1', 'Page 2', 'Page 3']

# POP — remove from top (simulates browser back button)
current = stack.pop()     # removes 'Page 3'
print('Back to:', stack[-1] if stack else 'No pages')  # Page 2

# PEEK — see top without removing
top = stack[-1]           # Page 2

# Check if empty
is_empty = len(stack) == 0

# Real-world: Undo feature in text editor
def text_editor():
    history = []
    text = ''
    actions = ['Type Hello', 'Type World', 'Undo', 'Type!']
    for action in actions:
        if action.startswith('Type '):
            word = action[5:]
            history.append(text)     # save state
```

```
            text += word + ' '
            print(f'Text: {text}')
        elif action == 'Undo' and history:
            text = history.pop()      # restore previous state
            print(f'Undo → Text: {text}')
text_editor()
```

16.2 Queue — First In, First Out (FIFO)

> **What is a Queue?**
> A queue works like a ticket counter line: first person in line is served first. The first item added is the first removed (FIFO).

```
from collections import deque

# Queue implementation using deque (double-ended queue)
queue = deque()

# ENQUEUE — add to back
queue.append('Customer 1')
queue.append('Customer 2')
queue.append('Customer 3')
print('Queue:', queue)    # deque(['Customer 1', 'Customer 2', 'Customer
3'])

# DEQUEUE — remove from front
served = queue.popleft()  # 'Customer 1' — first in, first out
print('Served:', served)
print('Remaining:', queue)

# Real-world: Print queue simulation
print_queue = deque()
documents = ['Report.pdf', 'Invoice.pdf', 'Letter.pdf']
for doc in documents:
    print_queue.append(doc)     # add to queue
    print(f'Queued: {doc}')

print('\nPrinting:')
while print_queue:
    printing = print_queue.popleft()
    print(f'Printing: {printing}')
```

16.3 Stack vs Queue

Feature	Stack (LIFO)	Queue (FIFO)
Order	Last in, First out	First in, First out

Add	append() to top	append() to rear
Remove	pop() from top	popleft() from front
Python	list.append/pop	deque.append/popleft
Use cases	Undo, recursion, DFS	Print queue, BFS, scheduling

Chapter 16

Modules & Packages

Organize, Reuse and Import Python Code

17. Modules & Packages

What is a Module?
A module is a single Python file (.py) containing variables, functions, and classes that can be imported into other programs. Modules enable code reuse and organization.

Real-World Analogy
A module is like a toolbox. You don't carry all your tools everywhere — you keep them organized in a box and pick only what you need for each job.

17.1 Creating & Using a Module

```
# FILE: math_utils.py (this is your module)
PI = 3.14159                    # constant variable

def area_circle(radius):
    '''Calculate area of circle.'''
    return PI * radius ** 2

def area_rectangle(length, width):
    '''Calculate area of rectangle.'''
    return length * width

def is_prime(n):
    '''Check if number is prime.'''
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True
```

```
# FILE: main.py (import and use the module)
import math_utils           # import entire module

print(math_utils.PI)        # 3.14159
print(math_utils.area_circle(7))        # 153.93...
print(math_utils.is_prime(17))          # True

# Import specific functions (more efficient)
```

```
from math_utils import area_rectangle, is_prime
print(area_rectangle(5, 3))  # 15

# Import with alias
import math_utils as mu
print(mu.area_circle(5))     # 78.53...
```

17.2 Built-in Python Modules

```
# math — mathematical functions
import math
print(math.sqrt(144))    # 12.0 — square root
print(math.ceil(4.2))    # 5    — round up
print(math.floor(4.9))   # 4    — round down
print(math.factorial(5)) # 120
print(math.pi)           # 3.14159...

# random — random number generation
import random
print(random.randint(1, 100))         # random int 1-100
print(random.choice(['a','b','c']))   # random element
items = [1, 2, 3, 4, 5]
random.shuffle(items)                 # shuffle in place

# os — operating system operations
import os
print(os.getcwd())                    # current directory
print(os.listdir('.'))                # list files
os.makedirs('new_folder', exist_ok=True)  # create folder

# datetime — date and time
from datetime import datetime, date, timedelta
now = datetime.now()
print(now.strftime('%d/%m/%Y %H:%M'))  # 17/03/2026 10:30
tomorrow = date.today() + timedelta(days=1)
print(tomorrow)
```

17.3 Creating a Package

```
# FOLDER STRUCTURE of a package:
# myapp/
# ├── __init__.py       (makes folder a package)
# ├── database.py       (database operations)
# ├── utils.py          (utility functions)
# └── models.py         (data models)

# FILE: myapp/__init__.py
# Can be empty or contain package-level code
```

```
print('myapp package loaded')

# FILE: myapp/utils.py
def validate_email(email):
    return '@' in email and '.' in email

def format_name(first, last):
    return f'{first.capitalize()} {last.capitalize()}'

# FILE: main.py — Using the package
from myapp.utils import validate_email, format_name
print(validate_email('student@school.com'))  # True
print(format_name('john', 'doe'))            # John Doe
```

Chapter 17

Exception Handling

Graceful Error Management with try/except/finally

18. Exception Handling

What is Exception Handling?
Exception handling is a mechanism to catch and manage runtime errors gracefully — preventing program crashes and providing meaningful error messages to users.

Real-World Analogy
Exception handling is like wearing a seatbelt. You can't always prevent accidents (errors), but the seatbelt (exception handler) ensures the impact is manageable and you don't crash fatally.

18.1 Basic try-except

```
# try — code that might raise an exception
# except — code to handle the exception

try:
    number = int(input('Enter a number: '))  # might fail
    result = 100 / number                    # might divide by zero
    print(f'100 / {number} = {result}')
except ValueError:
    print('Error: Please enter a valid integer!')
except ZeroDivisionError:
    print('Error: Cannot divide by zero!')
```

18.2 try-except-else-finally

```
# Full exception handling structure
try:
    a = int(input('Enter numerator: '))
    b = int(input('Enter denominator: '))
    result = a / b
except ValueError as e:          # 'as e' captures error message
    print(f'ValueError: {e}')
except ZeroDivisionError:
    print('Cannot divide by zero!')
else:
    # runs ONLY if no exception occurred
```

```
    print(f'Success! Result: {result:.2f}')
finally:
    # ALWAYS runs — for cleanup (close file, DB connection, etc.)
    print('Calculation attempt complete.')
```

18.3 Common Built-in Exceptions

Exception	Cause	Example
ValueError	Wrong value type	int('hello')
ZeroDivisionError	Divide by zero	10 / 0
TypeError	Wrong operation for type	'a' + 5
IndexError	List index out of range	[1,2][5]
KeyError	Key not in dictionary	d['bad']
FileNotFoundError	File doesn't exist	open('ghost.txt')
AttributeError	Invalid attribute access	'str'.append()
NameError	Variable not defined	print(x)
ImportError	Module not found	import ghost

18.4 Raising Exceptions

```
# raise — manually trigger an exception
def set_age(age):
    if not isinstance(age, int):
        raise TypeError('Age must be an integer')
    if age < 0 or age > 150:
        raise ValueError(f'Age {age} is not realistic')
    print(f'Age set to {age}')

try:
    set_age(-5)
except ValueError as e:
    print(f'Invalid age: {e}')

# Custom exception class
class InsufficientFundsError(Exception):
    def __init__(self, amount, balance):
        super().__init__(f'Cannot withdraw {amount}. Balance: {balance}')

def withdraw(amount, balance):
    if amount > balance:
        raise InsufficientFundsError(amount, balance)
    return balance - amount

try:
```

```
    new_balance = withdraw(5000, 3000)
except InsufficientFundsError as e:
    print(e)  # Cannot withdraw 5000. Balance: 3000
```

Chapter 18

File Handling in Python

Create, Read, Write, Append & Delete Files

19. File Handling

What is File Handling?
File handling lets Python programs create, read, write, and delete files on the computer. It is essential for data persistence — storing information between program runs.

19.1 File Modes

Mode	Description	Creates File?	Overwrites?
'r'	Read only (default)	No	No
'w'	Write (overwrites if exists)	Yes	Yes
'a'	Append (adds to end)	Yes	No
'x'	Exclusive create (fails if exists)	Yes	No
'rb'	Read binary	No	No
'wb'	Write binary	Yes	Yes

19.2 Writing & Reading Files

```
# WRITING a file — creates or overwrites
with open('students.txt', 'w') as f:    # 'with' auto-closes file
    f.write('Alice,20,Python\n')        # \n = newline
    f.write('Bob,21,Java\n')
    f.write('Charlie,22,C++\n')

# READING entire file
with open('students.txt', 'r') as f:
    content = f.read()                # read everything
    print(content)

# READING line by line (memory efficient for large files)
with open('students.txt', 'r') as f:
    for line in f:
        parts = line.strip().split(',')  # strip removes newline
        name, age, lang = parts
        print(f'{name} (Age {age}) studies {lang}')
```

```
# READING all lines as list
with open('students.txt', 'r') as f:
    lines = f.readlines()            # ['Alice,20,Python\n', ...]
    print(f'Total students: {len(lines)}')
```

19.3 Appending to Files

```
# APPENDING — adds to end without deleting existing content
with open('students.txt', 'a') as f:
    f.write('Diana,19,Python\n')     # adds new student
    f.write('Eve,20,Data Science\n')

print('New students added!')
```

19.4 File Operations with os Module

```
import os
import shutil

# Check if file exists before operations
file_path = 'students.txt'
if os.path.exists(file_path):
    size = os.path.getsize(file_path)
    print(f'File size: {size} bytes')

# Rename a file
os.rename('students.txt', 'class_roster.txt')

# Delete a file safely
if os.path.exists('class_roster.txt'):
    os.remove('class_roster.txt')
    print('File deleted')

# Create directory
os.makedirs('data/records', exist_ok=True)  # create nested dirs

# List all files in a directory
for file in os.listdir('.'):
    if file.endswith('.txt'):
        print(file)

# Real-world: Log system
def write_log(message, filename='app.log'):
    from datetime import datetime
    timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    with open(filename, 'a') as log:
        log.write(f'[{timestamp}] {message}\n')
```

```
write_log('Application started')
write_log('User Alice logged in')
write_log('Error: Database connection failed')
```

Chapter 19

Object-Oriented Programming

Classes, Objects, Inheritance, Polymorphism, Encapsulation, Abstraction

20. Object-Oriented Programming (OOP)

What is OOP?

OOP is a programming paradigm that organizes code around objects that combine data (attributes) and behavior (methods). Python is a fully object-oriented language. The 4 pillars of OOP are: Encapsulation, Inheritance, Polymorphism, and Abstraction.

Real-World Analogy

Think of a car blueprint (class). Every car manufactured from that blueprint is an object. The blueprint defines what properties (color, engine size) and behaviors (start, brake) every car will have.

20.1 Classes & Objects

```
# CLASS — the blueprint
class Student:
    # Class variable — shared by ALL objects
    school_name = 'Python Academy'

    # __init__ is the constructor — runs when object is created
    def __init__(self, name, age, gpa):
        # Instance variables — unique to each object
        self.name = name     # self refers to the current object
        self.age = age
        self.gpa = gpa
        self.courses = []    # empty list for each student

    # Instance method — behaviour of the object
    def introduce(self):
        return f'Hi! I am {self.name}, {self.age} yrs, GPA: {self.gpa}'

    def enroll(self, course):
        self.courses.append(course)
        print(f'{self.name} enrolled in {course}')

    def show_courses(self):
        print(f'{self.name} courses: {self.courses}')

    def __str__(self):   # string representation
        return f'Student({self.name}, GPA:{self.gpa})'
```

```
# Create OBJECTS (instances of the class)
s1 = Student('Alice', 20, 9.2)
s2 = Student('Bob', 21, 8.5)

# Use methods
print(s1.introduce())
s1.enroll('Python Programming')
s1.enroll('Data Structures')
s1.show_courses()

print(Student.school_name)    # class variable — same for all
print(s1)                     # calls __str__
```

20.2 Inheritance

What is Inheritance?
Inheritance allows a child class to acquire all properties and methods of a parent class, while also adding or overriding features. This promotes code reuse.

Single Inheritance

```
# Parent class
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def eat(self):
        print(f'{self.name} is eating.')

    def describe(self):
        print(f'{self.name} is a {self.species}')

# Child class inherits from Animal
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, 'Canine')  # call parent __init__
        self.breed = breed                # add child-specific attribute

    def bark(self):                       # child-specific method
        print(f'{self.name} says: Woof!')

    def describe(self):                   # METHOD OVERRIDING
        super().describe()                # call parent method
        print(f'Breed: {self.breed}')     # extend with more info

dog = Dog('Rex', 'German Shepherd')
dog.eat()       # inherited from Animal
dog.bark()      # Dog's own method
```

```
dog.describe()  # overridden method
```

Multiple Inheritance

```
# Multiple inheritance — inherits from more than one parent
class Flyable:
    def fly(self):
        print(f'{self.name} is flying!')

class Swimmable:
    def swim(self):
        print(f'{self.name} is swimming!')

class Duck(Animal, Flyable, Swimmable):  # inherits from 3 classes
    def __init__(self, name):
        super().__init__(name, 'Bird')

    def quack(self):
        print(f'{self.name}: Quack quack!')

duck = Duck('Donald')
duck.eat()    # from Animal
duck.fly()    # from Flyable
duck.swim()   # from Swimmable
duck.quack()  # Duck's own

# Method Resolution Order (MRO) — shows inheritance chain
print(Duck.mro())
```

Inheritance Types Summary

Type	Description	Example
Single	Child from one parent	Dog(Animal)
Multiple	Child from multiple parents	Duck(Animal, Flyable)
Multilevel	Grandparent → Parent → Child	Puppy(Dog(Animal))
Hierarchical	Multiple children, one parent	Cat(Animal), Dog(Animal)
Hybrid	Mix of multiple types	D(B, C) where B,C both from A

20.3 Polymorphism

What is Polymorphism?
Polymorphism means the same method name behaves differently for different objects. The most common form in Python is method overriding — where a child class redefines a parent class method.

```
# Polymorphism — same method, different behavior
class Shape:
    def area(self):
        return 0   # base implementation

    def describe(self):
        print(f'I am a {type(self).__name__} with area {self.area():.2f}')

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):        # OVERRIDE parent method
        return 3.14159 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

class Triangle(Shape):
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return 0.5 * self.base * self.height

# Polymorphism in action — same call, different behavior
shapes = [Circle(7), Rectangle(5, 4), Triangle(6, 8)]
for shape in shapes:
    shape.describe()   # each calls its OWN area()
```

20.4 Encapsulation

What is Encapsulation?
Encapsulation hides internal data from direct outside access. It wraps data and methods into a class and controls access using public, protected, and private modifiers.

```
class BankAccount:
    def __init__(self, owner, initial_balance):
        self.owner = owner               # PUBLIC — accessible anywhere
        self._bank_name = 'PyBank'       # PROTECTED — use inside
class/subclass
        self.__balance = initial_balance  # PRIVATE — only inside class
```

```
    # Getter — controlled read access to private data
    @property
    def balance(self):
        return self.__balance

    # Setter — controlled write access with validation
    @balance.setter
    def balance(self, amount):
        if amount < 0:
            raise ValueError('Balance cannot be negative')
        self.__balance = amount

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError('Deposit must be positive')
        self.__balance += amount
        print(f'Deposited {amount}. New balance: {self.__balance}')

    def withdraw(self, amount):
        if amount > self.__balance:
            raise ValueError('Insufficient funds!')
        self.__balance -= amount
        print(f'Withdrew {amount}. Remaining: {self.__balance}')

# Usage
acc = BankAccount('Alice', 5000)
print(acc.owner)          # Alice — public access
print(acc.balance)        # 5000 — via property getter
acc.deposit(1000)         # Deposited 1000. New balance: 6000
acc.withdraw(500)         # Withdrew 500. Remaining: 5500
# print(acc.__balance)  # ERROR — private attribute!
```

20.5 Abstraction

What is Abstraction?
Abstraction hides complex implementation details and exposes only what is necessary. In Python, abstraction is achieved using the ABC (Abstract Base Class) module.

```
from abc import ABC, abstractmethod

# Abstract class — cannot be instantiated directly
class PaymentGateway(ABC):
    @abstractmethod
    def process_payment(self, amount):   # MUST be implemented by
subclasses
        pass

    @abstractmethod
```

```
    def refund(self, transaction_id):
        pass

    def display_receipt(self, amount):   # Concrete method — shared by all
        print(f'Receipt: Paid Rs.{amount}')

# Concrete classes implement abstract methods
class UPIPayment(PaymentGateway):
    def process_payment(self, amount):
        print(f'Processing Rs.{amount} via UPI...')
        print('UPI payment successful!')

    def refund(self, tid):
        print(f'UPI Refund for transaction {tid} initiated')

class CreditCardPayment(PaymentGateway):
    def process_payment(self, amount):
        print(f'Charging Rs.{amount} to Credit Card...')
        print('Credit card payment approved!')

    def refund(self, tid):
        print(f'Credit card refund for {tid} in 3-5 days')

# Use polymorphically
payments = [UPIPayment(), CreditCardPayment()]
for p in payments:
    p.process_payment(500)
    p.display_receipt(500)   # inherited concrete method
```

Chapter 20

Advanced Python Concepts

Iterators, Generators, Closures & Decorators

21. Iterators & Generators

21.1 Iterators

Iterator
An iterator is an object that allows sequential access to elements one at a time using iter() and next() functions. When no more elements remain, it raises StopIteration.

```
# Using built-in iterator
my_list = [10, 20, 30, 40]
my_iter = iter(my_list)       # create iterator

print(next(my_iter))  # 10
print(next(my_iter))  # 20
print(next(my_iter))  # 30
print(next(my_iter))  # 40
# next(my_iter)       # StopIteration error — no more elements

# Custom iterator class
class CountDown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):    # makes object iterable
        return self

    def __next__(self):    # returns next value
        if self.current <= 0:
            raise StopIteration
        val = self.current
        self.current -= 1
        return val

for num in CountDown(5):   # 5, 4, 3, 2, 1
    print(num, end=' ')
```

21.2 Generators

Generator

A generator is a special function that yields values one at a time using the yield keyword. It saves memory because values are generated on demand — not stored all at once.

```
# Generator function using yield
def fibonacci_gen(limit):
    a, b = 0, 1
    while a <= limit:
        yield a           # pause and return value
        a, b = b, a + b   # resume from here next time

# Use like any iterable
for num in fibonacci_gen(100):
    print(num, end=' ')   # 0 1 1 2 3 5 8 13 21 34 55 89

# Generator expression — like list comprehension but lazy
squares_gen = (x**2 for x in range(1, 1000001))  # no memory waste
print(next(squares_gen))   # 1
print(next(squares_gen))   # 4

# Real-world: Read large log file line by line
def read_large_file(filepath):
    with open(filepath, 'r') as f:
        for line in f:
            yield line.strip()   # yield one line at a time

# Process without loading entire file into memory
# for log_entry in read_large_file('server.log'):
    # process(log_entry)
```

21.3 Decorators

```
# Decorator — wraps a function to add extra behaviour
import functools
import time

# 1. Timer decorator
def timer(func):
    @functools.wraps(func)      # preserves original function name
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - start
        print(f'{func.__name__} ran in {elapsed:.4f}s')
        return result
    return wrapper

@timer
def calculate_sum(n):
    return sum(range(n))
```

```
print(calculate_sum(1000000))

# 2. Login required decorator (web-style)
def login_required(func):
    @functools.wraps(func)
    def wrapper(user, *args, **kwargs):
        if not user.get('logged_in'):
            print('Access denied! Please login first.')
            return None
        return func(user, *args, **kwargs)
    return wrapper

@login_required
def view_dashboard(user):
    print(f'Welcome {user["name"]}! Here is your dashboard.')

view_dashboard({'name': 'Alice', 'logged_in': True})
view_dashboard({'name': 'Bob', 'logged_in': False})
```

Chapter 21

Regular Expressions (Regex)

Pattern Matching, Validation & Text Processing

22. Regular Expressions

What is Regex?
A regular expression (regex) is a pattern of characters used to search, match, or replace text. Python's re module provides regex support.

22.1 re Module Functions

Function	Purpose	Returns
`re.search(pattern, text)`	Find first match anywhere	Match object or None
`re.match(pattern, text)`	Match at START of string	Match object or None
`re.findall(pattern, text)`	Find ALL matches	List of strings
`re.sub(pattern, repl, text)`	Replace matches	New string
`re.split(pattern, text)`	Split by pattern	List of strings

22.2 Common Regex Patterns

Pattern	Meaning	Example Match
`\\d`	Any digit [0-9]	7, 3, 0
`\\D`	Any non-digit	a, !, space
`\\w`	Word char [a-zA-Z0-9_]	hello, 123
`\\W`	Non-word char	!, @, space
`\\s`	Whitespace	space, tab, newline
`\\S`	Non-whitespace	any visible char
`.`	Any char except newline	a, 5, @
`^`	Start of string	^Hello matches 'Hello!'
`$`	End of string	world$ matches 'hello world'
`*`	0 or more	ab* matches a, ab, abb

+	1 or more	ab+ matches ab, abb
?	0 or 1	ab? matches a, ab
{n}	Exactly n times	\\d{4} matches 2025
[abc]	Character class	[aeiou] matches vowels
(a|b)	OR	cat|dog matches cat or dog

22.3 Practical Regex Examples

```
import re

# 1. Validate email address
def validate_email(email):
    pattern = r'^[\w\.-]+@[\w\.-]+\.\w{2,4}$'
    return bool(re.match(pattern, email))

print(validate_email('student@college.edu'))   # True
print(validate_email('invalid-email'))         # False

# 2. Validate Indian mobile number
def validate_phone(phone):
    pattern = r'^[6-9]\d{9}$'   # starts 6-9, total 10 digits
    return bool(re.match(pattern, phone))

print(validate_phone('9876543210'))  # True
print(validate_phone('1234567890'))  # False

# 3. Extract all numbers from text
text = 'Order 42 items at Rs.999 with 18% GST'
numbers = re.findall(r'\d+', text)
print(numbers)   # ['42', '999', '18']

# 4. Remove extra spaces
messy = '  This   has    too   many   spaces  '
clean = re.sub(r'\s+', ' ', messy).strip()
print(clean)     # This has too many spaces

# 5. Password strength validator
def validate_password(pwd):
    if len(pwd) < 8: return 'Too short'
    if not re.search(r'[A-Z]', pwd): return 'Need uppercase'
    if not re.search(r'[a-z]', pwd): return 'Need lowercase'
    if not re.search(r'\d', pwd): return 'Need a digit'
    if not re.search(r'[@#$%&!]', pwd): return 'Need special char'
    return 'Strong password!'

print(validate_password('Abc@1234'))   # Strong password!
print(validate_password('weakpass'))   # Need uppercase

# 6. Extract dates from text
```

```
text = 'Event on 17/03/2026 and deadline is 25/03/2026'
dates = re.findall(r'\d{2}/\d{2}/\d{4}', text)
print(dates)   # ['17/03/2026', '25/03/2026']
```

Chapter 22

Threads & Multithreading

Concurrent execution, Thread Methods & Synchronization

23. Multithreading

What is a Thread?
A thread is the smallest unit of a process. Multithreading allows a program to run multiple threads concurrently, improving performance especially for I/O-bound tasks (file reads, network calls, database operations).

Real-World Analogy
While cooking a meal: one hand is boiling water (Thread 1), the other hand is chopping vegetables (Thread 2), and your eyes are watching the recipe (Thread 3). All tasks happen at the same time — that is multithreading.

23.1 Creating & Starting Threads

```
import threading
import time

def download_file(filename, size_mb):
    print(f'Starting download: {filename}')
    time.sleep(size_mb * 0.1)   # simulate download time
    print(f'Finished: {filename}')

# WITHOUT threading — sequential, one after another
start = time.time()
download_file('video.mp4', 50)
download_file('music.mp3', 5)
print(f'Sequential: {time.time()-start:.1f}s')

# WITH threading — both downloads happen at same time
start = time.time()
t1 = threading.Thread(target=download_file, args=('video.mp4', 50))
t2 = threading.Thread(target=download_file, args=('music.mp3', 5))

t1.start()    # start thread 1
t2.start()    # start thread 2

t1.join()     # wait for thread 1 to finish
t2.join()     # wait for thread 2 to finish
print(f'Threaded: {time.time()-start:.1f}s (much faster!)')
```

23.2 Thread Methods

Method	Description
`t.start()`	Start the thread's activity
`t.run()`	Define what thread does (override in subclass)
`t.join()`	Wait for thread to complete
`t.is_alive()`	Returns True if still running
`t.name`	Get/set thread name
`threading.current_thread()`	Get currently running thread

23.3 Thread Synchronization with Lock

```
import threading

# PROBLEM: Multiple threads accessing shared data causes race conditions
lock = threading.Lock()    # Lock prevents simultaneous access
balance = {'amount': 1000}

def withdraw(user, amount):
    lock.acquire()         # LOCK — only one thread at a time
    try:
        if balance['amount'] >= amount:
            print(f'{user}: Withdrawing Rs.{amount}')
            balance['amount'] -= amount
            print(f'Balance left: Rs.{balance["amount"]}')
        else:
            print(f'{user}: Insufficient funds!')
    finally:
        lock.release()     # ALWAYS release the lock

# Two people try to withdraw simultaneously
t1 = threading.Thread(target=withdraw, args=('Alice', 800))
t2 = threading.Thread(target=withdraw, args=('Bob', 500))
t1.start()
t2.start()
t1.join()
t2.join()

# With-statement (cleaner way to use lock)
def safe_deposit(amount):
    with lock:             # auto-acquires and releases
        balance['amount'] += amount
        print(f'Deposited Rs.{amount}. Balance: Rs.{balance["amount"]}')
```

Chapter 23

Python Database (MySQL)

Connect, CRUD Operations & Real-World Examples

24. Python with MySQL Database

Why use Python with MySQL?
MySQL is a popular open-source relational database. Python's mysql-connector-python library lets you connect your programs to databases to store, retrieve, update, and delete data.

24.1 Setup & Connection

```
# Step 1: Install the connector
# pip install mysql-connector-python

# Step 2: Import and connect
import mysql.connector

def get_connection():
    '''Create and return a database connection.'''
    return mysql.connector.connect(
        host='localhost',
        user='root',
        password='your_password',
        database='school_db'
    )

# Test connection
try:
    conn = get_connection()
    print('Connected to MySQL!')
    conn.close()
except mysql.connector.Error as e:
    print(f'Connection failed: {e}')
```

24.2 Create Database & Table

```
import mysql.connector

conn = mysql.connector.connect(host='localhost', user='root',
password='raja')
cursor = conn.cursor()
```

```
# Create database
cursor.execute('CREATE DATABASE IF NOT EXISTS school_db')
conn.database = 'school_db'   # switch to this database

# Create table
cursor.execute('''
    CREATE TABLE IF NOT EXISTS students (
        id          INT AUTO_INCREMENT PRIMARY KEY,
        name        VARCHAR(100) NOT NULL,
        age         INT,
        course      VARCHAR(50),
        marks       FLOAT,
        created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )
''')

conn.commit()   # save changes
print('Database and table created!')
conn.close()
```

24.3 INSERT Data

```
# INSERT single record
conn =
mysql.connector.connect(host='localhost',user='root',password='raja',database='school_db')
cursor = conn.cursor()

sql = 'INSERT INTO students (name, age, course, marks) VALUES (%s, %s, %s, %s)'
val = ('Alice', 20, 'Python', 92.5)
cursor.execute(sql, val)      # use parameterized queries — prevents SQL injection
conn.commit()
print(f'Inserted ID: {cursor.lastrowid}')

# INSERT multiple records
students_data = [
    ('Bob', 21, 'Java', 88.0),
    ('Charlie', 22, 'Python', 95.5),
    ('Diana', 20, 'C++', 79.0),
    ('Eve', 23, 'Python', 91.0),
]
cursor.executemany(sql, students_data)
conn.commit()
print(f'Inserted {cursor.rowcount} records')
conn.close()
```

24.4 SELECT (Read) Data

```
conn =
mysql.connector.connect(host='localhost',user='root',password='raja',database='school_db')
cursor = conn.cursor()

# SELECT all records
cursor.execute('SELECT * FROM students')
records = cursor.fetchall()
for row in records:
    print(row)

# SELECT with WHERE condition
cursor.execute('SELECT name, marks FROM students WHERE course = %s', ('Python',))
python_students = cursor.fetchall()
for name, marks in python_students:
    print(f'{name}: {marks}')

# SELECT with ORDER BY
cursor.execute('SELECT name, marks FROM students ORDER BY marks DESC LIMIT 3')
top3 = cursor.fetchall()
print('Top 3 students:')
for i, (name, marks) in enumerate(top3, 1):
    print(f'{i}. {name}: {marks}')
conn.close()
```

24.5 UPDATE & DELETE

```
conn =
mysql.connector.connect(host='localhost',user='root',password='raja',database='school_db')
cursor = conn.cursor()

# UPDATE a record
cursor.execute('UPDATE students SET marks = %s WHERE name = %s', (98.0, 'Alice'))
conn.commit()
print(f'{cursor.rowcount} record updated')

# DELETE a record
cursor.execute('DELETE FROM students WHERE name = %s', ('Bob',))
conn.commit()
print(f'{cursor.rowcount} record deleted')

# DROP a table
# cursor.execute('DROP TABLE IF EXISTS students')

conn.close()
```

Chapter 24

Network Programming

Socket Programming, Client-Server & Chat App

25. Network Programming

What is Network Programming?
Network programming involves writing code that allows computers to communicate over a network. Python's socket module provides low-level network interface for TCP/UDP communication.

Real-World Analogy
A socket is like a phone call: one side dials (client connects), communication happens (data transfer), and then one side hangs up (connection closes).

25.1 Basic Server

```
# SERVER CODE — save as server.py and run first
import socket

def start_server(host='127.0.0.1', port=65432):
    # Create TCP socket (AF_INET=IPv4, SOCK_STREAM=TCP)
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    server.bind((host, port))    # bind to address and port
    server.listen(5)             # listen for up to 5 connections
    print(f'Server listening on {host}:{port}')

    while True:
        conn, addr = server.accept()   # wait for client
        print(f'Connected: {addr}')

        data = conn.recv(1024).decode()  # receive data
        print(f'Received: {data}')

        response = f'Server received: {data.upper()}'
        conn.sendall(response.encode())  # send response
        conn.close()                     # close client connection

start_server()
```

25.2 Basic Client

```
# CLIENT CODE — save as client.py and run after server
import socket

def connect_to_server(host='127.0.0.1', port=65432):
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((host, port))    # connect to server

    message = 'Hello from Python client!'
    client.sendall(message.encode())

    response = client.recv(1024).decode()
    print(f'Server replied: {response}')

    client.close()

connect_to_server()
```

25.3 Multi-Client Server (Threaded)

```
# Handles multiple clients simultaneously using threads
import socket, threading

def handle_client(conn, addr):
    print(f'New client: {addr}')
    try:
        while True:
            data = conn.recv(1024).decode()
            if not data or data.lower() == 'quit':
                break
            print(f'[{addr[1]}] {data}')
            conn.sendall(f'Echo: {data}'.encode())
    finally:
        conn.close()
        print(f'Client {addr} disconnected')

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(('127.0.0.1', 65432))
server.listen(10)
print('Multi-client server running...')

while True:
    conn, addr = server.accept()
    thread = threading.Thread(target=handle_client, args=(conn, addr))
    thread.daemon = True   # thread dies when main program exits
    thread.start()
```

Chapter 25

GUI in Python with Tkinter

Widgets, Events, Forms & Real-World Applications

26. GUI Programming with Tkinter

What is GUI?
A Graphical User Interface (GUI) lets users interact with programs visually using buttons, text boxes, labels, and menus. Python's built-in Tkinter library makes it easy to create desktop GUI applications.

26.1 Tkinter Basics

```
# Every Tkinter app starts with these 4 lines
from tkinter import *

root = Tk()                      # create main window
root.title('My Python App')      # set window title
root.geometry('500x400')         # set width x height
root.resizable(True, True)       # allow resizing
root.mainloop()                  # start the event loop (keeps window open)
```

26.2 Common Widgets

Widget	Purpose	Example Use
Label	Display text/image	App title, instructions
Button	Clickable action	Submit, Login, Search
Entry	Single-line text input	Name, email, password
Text	Multi-line text area	Description, comments
Checkbutton	True/False checkbox	Accept terms, preferences
Radiobutton	Single choice from many	Gender, payment method
Combobox	Dropdown selection	Country, language
Frame	Container for widgets	Group related widgets
Canvas	Draw shapes/images	Charts, drawing app
MessageBox	Popup dialogs	Alert, confirm, error

26.3 Complete Login Form

```
from tkinter import *
from tkinter import messagebox

def login():
    username = username_entry.get()    # get text from Entry widget
    password = password_entry.get()

    # Simple validation
    if not username or not password:
        messagebox.showwarning('Warning', 'Please fill all fields!')
        return

    if username == 'admin' and password == '1234':
        messagebox.showinfo('Success', f'Welcome, {username}!')
    else:
        messagebox.showerror('Error', 'Invalid credentials!')

def clear_form():
    username_entry.delete(0, END)   # clear Entry widget
    password_entry.delete(0, END)
    username_entry.focus()          # set cursor focus

# Main window
root = Tk()
root.title('Login Form')
root.geometry('350x250')
root.configure(bg='#f0f0f0')

# Title label
Label(root, text='Python Login', font=('Arial', 16, 'bold'),
      bg='#f0f0f0', fg='#1565C0').pack(pady=20)

# Username
Label(root, text='Username:', bg='#f0f0f0').pack()
username_entry = Entry(root, width=25, font=('Arial', 11))
username_entry.pack(pady=5)

# Password
Label(root, text='Password:', bg='#f0f0f0').pack()
password_entry = Entry(root, width=25, show='*', font=('Arial', 11))
password_entry.pack(pady=5)

# Buttons
btn_frame = Frame(root, bg='#f0f0f0')
btn_frame.pack(pady=15)
Button(btn_frame, text='Login', command=login, bg='#1565C0',
       fg='white', width=10).pack(side=LEFT, padx=5)
Button(btn_frame, text='Clear', command=clear_form,
       width=10).pack(side=LEFT, padx=5)

root.mainloop()
```

26.4 Simple Calculator GUI

```
from tkinter import *

def button_click(value):
    current = display.get()         # get current text
    display.delete(0, END)
    display.insert(END, current + str(value))

def calculate():
    try:
        result = eval(display.get())  # evaluate expression
        display.delete(0, END)
        display.insert(END, result)
    except:
        display.delete(0, END)
        display.insert(END, 'Error')

def clear():
    display.delete(0, END)

root = Tk()
root.title('Calculator')
root.geometry('300x400')
root.resizable(False, False)

# Display
display = Entry(root, font=('Arial', 18), bd=5, justify=RIGHT)
display.grid(row=0, column=0, columnspan=4, sticky='we', padx=5, pady=5)

# Button layout
buttons = [
    ['7','8','9','/'],
    ['4','5','6','*'],
    ['1','2','3','-'],
    ['0','.','=','+'],
]

for r, row in enumerate(buttons, 1):
    for c, btn in enumerate(row):
        cmd = calculate if btn == '=' else lambda v=btn: button_click(v)
        Button(root, text=btn, width=7, height=2, font=('Arial',12),
               command=cmd).grid(row=r, column=c, padx=2, pady=2)

Button(root, text='C', width=15, height=2, bg='red', fg='white',
       command=clear).grid(row=5, column=0, columnspan=2, padx=2, pady=2)

root.mainloop()
```

26.5 Student Registration Form

```
from tkinter import *
from tkinter import ttk, messagebox

def submit():
    data = {
        'name':   name_var.get(),
        'age':    age_var.get(),
        'gender': gender_var.get(),
        'course': course_combo.get(),
        'agree':  agree_var.get()
    }
    if not all([data['name'], data['age'], data['gender'],
data['course']]):
        messagebox.showwarning('Warning', 'Fill all required fields!')
        return
    if not data['agree']:
        messagebox.showwarning('Warning', 'Please accept the terms!')
        return
    summary = '\n'.join([f'{k.title()}: {v}' for k, v in data.items()])
    messagebox.showinfo('Registration Successful', summary)

root = Tk()
root.title('Student Registration')
root.geometry('400x350')

name_var, age_var, gender_var, agree_var = StringVar(), StringVar(),
StringVar(), BooleanVar()

fields = [('Full Name', name_var), ('Age', age_var)]
for i, (label, var) in enumerate(fields):
    Label(root, text=label).grid(row=i, column=0, sticky='e', padx=10,
pady=5)
    Entry(root, textvariable=var, width=25).grid(row=i, column=1, pady=5)

Label(root, text='Gender').grid(row=2, column=0, sticky='e', padx=10)
for i, g in enumerate(['Male','Female','Other']):
    Radiobutton(root, text=g, variable=gender_var, value=g).grid(row=2,
column=1+i)

Label(root, text='Course').grid(row=3, column=0, sticky='e', padx=10,
pady=5)
course_combo = ttk.Combobox(root, values=['Python','Java','C++','Data
Science'], width=22)
course_combo.grid(row=3, column=1, pady=5)

Checkbutton(root, text='I agree to terms', variable=agree_var).grid(row=4,
column=1, sticky='w')
Button(root, text='Register', command=submit, bg='green', fg='white',
width=15).grid(row=5, column=1, pady=10)

root.mainloop()
```

Chapter 26

Real-World Project Code

Complete runnable projects combining multiple concepts

27. Real-World Project Examples

27.1 Student Grade Management System

```
# Complete Student Grade Management System
# Covers: OOP, File Handling, Exception Handling, Data Structures

import json, os

class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = {}   # {'subject': grade}

    def add_grade(self, subject, grade):
        if not 0 <= grade <= 100:
            raise ValueError(f'Grade {grade} must be 0-100')
        self.grades[subject] = grade

    def average(self):
        return sum(self.grades.values()) / len(self.grades) if self.grades
else 0

    def letter_grade(self):
        avg = self.average()
        if avg >= 90: return 'A+'
        if avg >= 80: return 'A'
        if avg >= 70: return 'B'
        if avg >= 60: return 'C'
        return 'F'

    def report(self):
        print(f'\n{'-'*40}')
        print(f'Student: {self.name} (ID: {self.student_id})')
        for subject, grade in self.grades.items():
            print(f'  {subject}: {grade}')
        print(f'  Average: {self.average():.1f} ({self.letter_grade()})')

class GradeBook:
    def __init__(self, filename='grades.json'):
        self.students = {}
        self.filename = filename
```

```
        self.load()        # load from file on startup

    def add_student(self, sid, name):
        if sid in self.students:
            raise ValueError(f'Student {sid} already exists')
        self.students[sid] = Student(sid, name)
        print(f'Added: {name}')

    def add_grade(self, sid, subject, grade):
        if sid not in self.students:
            raise KeyError(f'Student {sid} not found')
        self.students[sid].add_grade(subject, grade)
        self.save()

    def top_students(self, n=3):
        ranked = sorted(self.students.values(), key=lambda s: s.average(),
reverse=True)
        return ranked[:n]

    def save(self):
        data = {sid: {'name': s.name, 'grades': s.grades}
                for sid, s in self.students.items()}
        with open(self.filename, 'w') as f:
            json.dump(data, f, indent=2)

    def load(self):
        if os.path.exists(self.filename):
            with open(self.filename, 'r') as f:
                data = json.load(f)
            for sid, info in data.items():
                s = Student(sid, info['name'])
                s.grades = info['grades']
                self.students[sid] = s

# ---- Main program ----
book = GradeBook()
book.add_student('S001', 'Alice')
book.add_student('S002', 'Bob')
book.add_student('S003', 'Charlie')

book.add_grade('S001', 'Math', 92)
book.add_grade('S001', 'Science', 88)
book.add_grade('S002', 'Math', 75)
book.add_grade('S002', 'Science', 82)
book.add_grade('S003', 'Math', 95)
book.add_grade('S003', 'Science', 91)

for s in book.students.values():
    s.report()

print('\nTop Students:')
for i, s in enumerate(book.top_students(), 1):
    print(f'{i}. {s.name}: {s.average():.1f}')
```

27.2 Password Manager

```
# Secure Password Manager
# Covers: OOP, File Handling, Exception Handling, Sets/Dicts

import json, os, random, string
from datetime import datetime

class PasswordManager:
    def __init__(self, vault_file='vault.json'):
        self.vault = {}
        self.vault_file = vault_file
        self.load_vault()

    def generate_password(self, length=12, use_symbols=True):
        chars = string.ascii_letters + string.digits
        if use_symbols:
            chars += '!@#$%&*'
        return ''.join(random.choice(chars) for _ in range(length))

    def strength_check(self, password):
        checks = {
            'length':   len(password) >= 8,
            'upper':    any(c.isupper() for c in password),
            'lower':    any(c.islower() for c in password),
            'digit':    any(c.isdigit() for c in password),
            'symbol':   any(c in '!@#$%&*' for c in password),
        }
        strength = sum(checks.values())
        label = ['Very Weak','Weak','Moderate','Strong','Very
Strong'][strength-1]
        return label, checks

    def save_entry(self, site, username, password):
        self.vault[site] = {
            'username': username,
            'password': password,
            'added':    datetime.now().strftime('%Y-%m-%d'),
        }
        self.save_vault()
        print(f'Saved credentials for {site}')

    def get_entry(self, site):
        return self.vault.get(site, None)

    def save_vault(self):
        with open(self.vault_file, 'w') as f:
            json.dump(self.vault, f, indent=2)

    def load_vault(self):
        if os.path.exists(self.vault_file):
```

```
            with open(self.vault_file, 'r') as f:
                self.vault = json.load(f)

pm = PasswordManager()
pwd = pm.generate_password(16)
print(f'Generated: {pwd}')
strength, checks = pm.strength_check(pwd)
print(f'Strength: {strength}')
pm.save_entry('github.com', 'alice', pwd)
entry = pm.get_entry('github.com')
print(f'Stored: {entry}')
```

PYTHON QUICK REFERENCE CHEATSHEET

All Key Syntax & Concepts in One Place

PYTHON CHEATSHEET

Variables & Types

```
x = 10          # int       | pi = 3.14      # float
s = 'hello'     # str       | b = True        # bool
lst = [1,2,3]   # list      | t = (1,2)       # tuple (immutable)
d = {'k':'v'}   # dict      | s = {1,2,3}     # set (unique)
type(x)  int()  float()  str()  bool()  list()  tuple()  dict()  set()
```

Operators Quick Reference

```
# Arithmetic:  +  -  *  /  //  %  **
# Comparison:  ==  !=  >  <  >=  <=
# Logical:     and  or  not
# Assignment:  =  +=  -=  *=  /=  //=  %=  **=
# Identity:    is  is not
# Membership:  in  not in
```

Control Flow

```
if x > 0:         while x > 0:        for i in range(5):
    pass          x -= 1              print(i)
elif x == 0:      else:               for k,v in d.items():
    pass              print('done')      print(k,v)
else:
    pass
# Ternary: val = 'yes' if x > 0 else 'no'
# match-case: match x: case 1: ... case _: ...
# break | continue | pass
```

Functions

```
def func(pos, kw='default', *args, **kwargs):  return val
lambda x, y: x + y          # anonymous function
map(func, iterable)          filter(func, iterable)
from functools import reduce; reduce(func, iterable)
```

Strings

```
s[0]  s[-1]  s[1:4]  s[::-1]  s*3  s+t  f'{var}'
.upper() .lower() .strip() .replace() .split() .join()
.find() .count() .startswith() .endswith() .isdigit()
```

Lists

```
.append(x) .extend([]) .insert(i,x) .remove(x) .pop(i)
.sort() .reverse() .index(x) .count(x) .copy() .clear()
Slicing: lst[1:4] | Comprehension: [x**2 for x in lst if x>0]
```

Dictionary

```
d[k]=v  d.get(k,'default')  d.keys()  d.values()  d.items()
d.update({})  d.pop(k)  del d[k]  k in d  d.clear()
{k:v for k,v in items}  # dict comprehension
```

OOP

```
class MyClass(Parent):
    class_var = 0
    def __init__(self, x): self.x = x
    def method(self): return self.x
    @classmethod    def cm(cls): pass
    @staticmethod   def sm(): pass
    @property       def prop(self): return self._x
super().__init__()  # call parent constructor
from abc import ABC, abstractmethod  # abstraction
```

Exception Handling

```
try:                        # raise ValueError('msg')
    risky_code()
except (ValueError, TypeError) as e:   print(e)
else:   print('no error')    # runs only if no exception
finally: cleanup()          # ALWAYS runs
```

File Handling

```
with open('file.txt', 'r') as f:   f.read()  f.readlines()
with open('file.txt', 'w') as f:   f.write('text')
with open('file.txt', 'a') as f:   f.write('append')
import os; os.path.exists() os.remove() os.makedirs() os.listdir()
```

Regex

```
import re
re.search(r'\d+', text)   re.findall(r'\w+', text)
re.sub(r'\s+', ' ', text) re.match(r'^Hello', text)
Patterns: \d=digit  \w=word  \s=space  .=any  ^start  $end
         +  *  ?  {n}  [abc]  (a|b)
```

Threading

```
import threading
t = threading.Thread(target=func, args=(x,), name='t1')
t.start()  t.join()  t.is_alive()
lock = threading.Lock()
with lock:  critical_section()  # thread synchronization
```

MySQL Database

```
import mysql.connector
conn = mysql.connector.connect(host=,user=,password=,database=)
cursor = conn.cursor()
cursor.execute('SQL', (params,))   cursor.executemany(sql, list)
cursor.fetchall()  cursor.fetchone()  conn.commit()  conn.close()
```

Tkinter GUI

```
from tkinter import *; root=Tk(); root.title(); root.geometry()
Label  Entry  Button  Text  Checkbutton  Radiobutton  Combobox
.pack()  .grid(row=,column=)  .place(x=,y=)
widget.get()  widget.delete(0,END)  widget.insert(END,'text')
root.mainloop()  # start event loop
```

IMPORTANT POINTS TO REMEMBER

Important Points

Python Fundamentals

- Python is CASE SENSITIVE: name != Name != NAME
- Python uses INDENTATION (4 spaces) instead of {} for code blocks
- Everything in Python is an OBJECT — even functions and classes
- Variables are REFERENCES to objects, not boxes containing objects
- Python has DYNAMIC TYPING — no need to declare variable types
- None is Python's null — not 0, not '', not False

Data Structures

- List: ordered, mutable [] | Tuple: ordered, immutable ()
- Dict: key-value, mutable {} | Set: unordered, unique {}
- Strings are IMMUTABLE — every 'change' creates a new string
- list.append() adds one item | list.extend() adds many items
- Use d.get(key, default) instead of d[key] to avoid KeyError
- Set removes duplicates automatically — great for deduplication

Functions & OOP

- default arguments must come AFTER positional arguments
- *args = variable positional args (tuple) | **kwargs = keyword args (dict)
- Lambda is for simple one-liners only — use def for complex logic
- Every class method's first parameter is self (refers to the object)
- __init__ is called automatically when creating an object
- super() calls the parent class method
- Private: __name (double underscore) | Protected: _name (single underscore)
- Abstract classes (ABC) cannot be instantiated directly

Error Handling & File Handling

- ALWAYS use try-except for user input and file operations
- finally block ALWAYS runs — use it for cleanup (close files/DB)
- Use 'with open()' instead of manual open/close — auto-closes
- 'w' mode OVERWRITES the file | 'a' mode APPENDS to the file
- Use parameterized queries (%s) in SQL to prevent SQL injection

Performance & Best Practices

- List comprehensions are faster and more Pythonic than for loops
- Use generators for large data — they save memory
- join() is faster than + for concatenating many strings
- Use threading for I/O-bound tasks, multiprocessing for CPU-bound
- Always validate and sanitize user input before processing
- Follow PEP 8 style guide: snake_case for variables and functions